

## TAREADESISTEMASEMBEBIDOS-TEÓRICO

### TAREA #4

#### OLED – DF PLAYER MINI

### 1. OBJETIVO GENERAL

- Desarrollar un juguete interactivo basado en el microcontrolador ESP32 que integre una pantalla OLED, un módulo DFPlayer Mini para reproducción de sonidos, y diferentes elementos de interacción como botones y potenciómetros, fomentando el diseño de sistemas embebidos aplicados a productos lúdicos.

### 2. OBJETIVOS ESPECÍFICOS

- Integrar el módulo DFPlayer Mini dentro de un sistema embebido para reproducir sonidos en respuesta a la interacción del usuario.
- Implementar una pantalla OLED que muestre información relevante del juguete (estados, animaciones, indicadores, etc.).
- Configurar botones, potenciómetros u otros sensores para permitir la interacción directa del usuario con el juguete.

### 3. INTRODUCCIÓN ADC y Señales Analógicas

En el mundo moderno, la mayor parte de la información se transmite de manera digital entre dispositivos electrónicos, no obstante, gran parte de esta información es originalmente analógica. Las señales analógicas captadas mediante sensores varían constantemente y a diferentes velocidades, pasando de velocidades lentas en señales como la temperatura a velocidades rápidas como una señal de audio. El inconveniente surge cuando estas señales son muy difíciles de manipular e inclusive de guardar y recuperar con exactitud, de modo que como solución a este problema surge el ADC.



*Ilustración 1. Convertidor analógico a digital.*

El ADC (Analog to Digital Converter) o Conversor analógico a digital es un circuito integrado que partiendo de una señal analógica puede obtener una señal digital equivalente que una máquina puede interpretar, manipular y guardar fácilmente. La exactitud con la que esta señal se convierte depende de la resolución del ADC y eso suele ser diferente en los microcontroladores.

La resolución del ADC se entiende como el voltaje necesario (señal analógica) para lograr que en la salida digital haya un cambio en el bit menos significativo. En otras

palabras, el voltaje (en señal digital) obtenido por unidad de voltaje en señal analógica.

$$V_{ref}$$

$$resolucion = 2_n - 1$$

Mientras mayor sea la resolución, más preciso sería la conversión obtenida por el ADC. En el caso del ESP32, el número de bits por defecto es de 12 bits, superior a la resolución del Atmega328P (Arduino) cuyo número de bits para el ADC es de solo 8 bits, sin embargo, se puede configurar el ADC para que trabaje con una resolución de 9 bits. Estos 12 bits por defecto del ESP32 indican que el rango de lectura de las entradas analógicas varía entre 0 y 4095, a diferencia del Atmega328P que varían del 0 al 255. Esta información es muy importante para obtener qué cantidad de voltaje estaríamos recibiendo como señal digital si recibimos X cantidad de señal analógica.

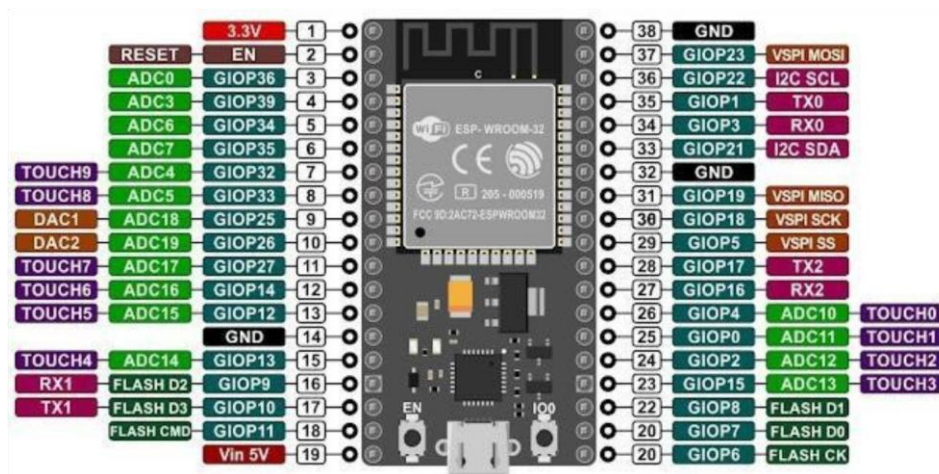


Ilustración2.PinoutESP32.

El ESP32 cuenta con 15 pines ADC y para hacer uso de cada uno basta con inicializar el pin a utilizar como entrada y colocar este pin dentro de la función **analogRead(pin)**, esta función retorna un valor entero de 0 a 4095 (cuando se usa el ADC por defecto) que correspondería a la lectura analógica.

### Pantalla OLED

Las pantallas OLED (Organic Light Emitting Diode) son dispositivos de visualización de alta calidad que se diferencian de las pantallas LCD tradicionales en su tecnología de emisión de luz. Cada píxel en una pantalla OLED emite su propia luz, lo que permite un mejor contraste, colores más vivos y un menor consumo de energía cuando se muestran imágenes oscuras. Gracias a estas características, las pantallas OLED son ampliamente utilizadas en sistemas embebidos, donde la eficiencia y la claridad de visualización son críticas.

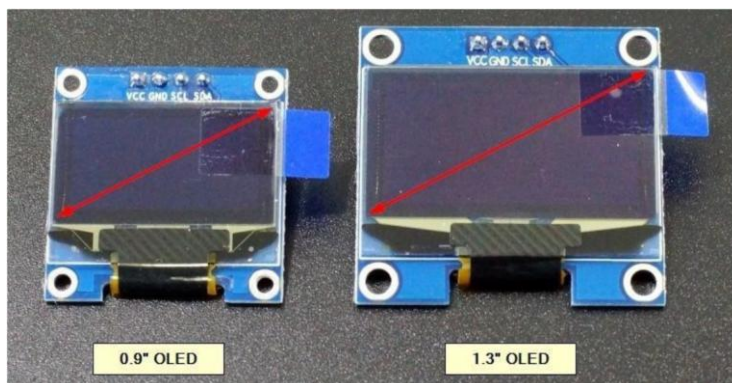


Ilustración3. PantallasOLEDmáscomerciales.

### Variedad de Tamaños

Las pantallas OLED se encuentran disponibles en una amplia variedad de tamaños, desde pequeños paneles de 0.49” hasta modelos mayores de 3.2” o más. En el caso de los sistemas embebidos, las pantallas de 0.96” son especialmente populares debido a su tamaño compacto y a la cantidad adecuada de información que pueden mostrar. Estas pantallas suelen tener una resolución de 128x64 o 128x32 píxeles, lo que permite representar texto, gráficas y otros elementos visuales de manera clara y eficiente.



Ilustración4. Ejemplificación de la variedad de tamaños de pantallas OLED en el mercado.

### Tipos de Comunicación Serial

Las pantallas OLED para sistemas embebidos generalmente utilizan dos tipos principales de interfaces de comunicación serial:

1. **I2C (Inter-Integrated Circuit):** Esta es una de las interfaces más comunes debido a su simplicidad y eficiencia. Solo requiere dos líneas de comunicación: SDA (datos) y SCL (reloj). Es ideal para aplicaciones donde se necesita conectar varios dispositivos al mismo bus.
2. **SPI (Serial Peripheral Interface):** Esta interfaz ofrece una velocidad de transmisión más alta en comparación con I2C. Requiere cuatro líneas de comunicación: MOSI (datos hacia la pantalla), MISO (datos hacia el controlador, aunque en algunas pantallas OLED no se usa), SCK (reloj) y CS

(selección de chip). SPI es preferido en aplicaciones donde la velocidad de actualización de la pantalla es un factor crítico.

### Configuración de Pines

Las pantallas OLED de 0.96" pueden venir con configuraciones de **4 pines** o **6 pines**, dependiendo del tipo de comunicación que soporten. Por ello, no se asuste muchas pantallas OLED vienen con 6 pines y están diseñadas para ser **interfases híbridas**, lo que significa que pueden configurarse para comunicarse tanto en **I2C** como en **SPI**, dependiendo de cómo las configures. Sin embargo, esto depende del modelo y del controlador específico de la pantalla (por ejemplo, el controlador SSD1306 que se encuentra en muchas pantallas OLED). De todas maneras, aquí se indican como conectarlas:

#### 1. Pantallas de 4 Pines:

- VCC: Alimentación
- (generalmente entre 3.3V y 5V).
- GND: Tierra.
- SDA:

Línea de datos (en comunicación I2C).

- SCL: Línea de reloj (en comunicación I2C). Esta configuración es típica para pantallas que usan el protocolo I2C, lo que simplifica el cableado y la conexión al microcontrolador.

#### 2. Pantallas de 6 Pines:

- VCC: Alimentación (3.3V o 5V).
- GND: Tierra.
- CS: Selección de chip (en comunicación SPI).

- RST: Reinicio de la pantalla.
- D/C: Selección de modo (datos o comando).
- SCK y MOSI: Líneas de reloj y datos respectivamente (en comunicación SPI). Esta configuración es común en pantallas que soportan SPI, proporcionando una mayor velocidad de comunicación a costa de un cableado más complejo.

**Importante:** Habría que revisar el datasheet de su pantalla OLED para determinar que pines representa D0 y D1, regularmente son SCK y SDA respectivamente.

### Conexión de Pantallas OLED

La conexión de una pantalla OLED a un sistema embebido depende del tipo de comunicación que utilice:

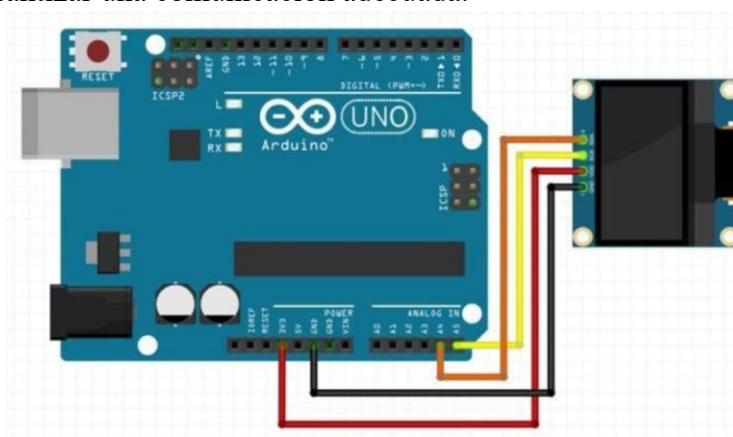
#### 1. Conexión mediante I2C:

- Conectar VCC y GND a la fuente de alimentación del microcontrolador.

- Conectar SDA y SCL a los pines correspondientes de comunicación I2C del microcontrolador.
- Configurar la dirección I2C en el código (generalmente 0x3C o 0x3D).

## 2. Conexión mediante SPI:

- Conectar VCC y GND a la fuente de alimentación.
- Conectar CS, RST, D/C, SCK y MOSI a los pines correspondientes del microcontrolador.
- Configurar la velocidad de reloj y el modo de SPI en el código para garantizar una comunicación adecuada.



*Ilustración5.EjemplodeconexiónI2C.*

## Importancia de las Pantallas OLED en los Sistemas Embebidos

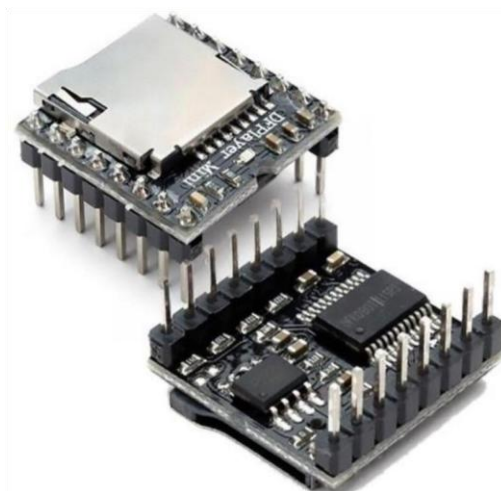
Las pantallas OLED son componentes esenciales en los sistemas embebidos debido a su capacidad para mostrar información en tiempo real de manera clara y eficiente. Una de sus principales ventajas es su consumo energético eficiente, ya que únicamente los píxeles encendidos consumen energía, lo que las convierte en una opción ideal para dispositivos alimentados por batería. Además, ofrecen un alto contraste que garantiza una excelente visibilidad incluso en condiciones de poca luz, lo que las hace particularmente útiles en entornos donde la legibilidad es crucial.

Su diseño compacto y ligero es otra característica destacada, ya que facilita su integración en dispositivos pequeños sin comprometer el espacio o la funcionalidad. Asimismo, estas pantallas ofrecen flexibilidad en el diseño, gracias a su capacidad de utilizar diferentes protocolos de comunicación y resoluciones. Esto permite adaptarlas a una amplia variedad de aplicaciones, desde dispositivos portátiles hasta sistemas industriales avanzados.

## Introducción al DFPlayer Mini

El DFPlayer Mini es un módulo de reproducción de audio compacto y de bajo costo que permite reproducir archivos de audio en formato MP3 almacenados en tarjetas microSD. Este módulo es ampliamente utilizado en proyectos de sistemas embebidos y

electrónica debido a su facilidad de uso y versatilidad para aplicaciones que requieren salida de audio.



*Ilustración6. MóduloDFPlayerMini.*

### Características Principales

El DFPlayer Mini es un módulo de reproducción de audio que sobresale por sus múltiples características que lo hacen ideal para una amplia variedad de aplicaciones. Entre sus principales atributos se encuentra su compatibilidad con diversos formatos de audio, incluyendo MP3, WAV y WMA, lo que le otorga gran versatilidad para manejar distintos tipos de contenido sonoro.

En cuanto al almacenamiento, el dispositivo admite tarjetas microSD, microSDHC y TF con capacidades de hasta 32 GB, ofreciendo una solución flexible y práctica para el almacenamiento de archivos de audio. Su interfaz de comunicación es otro punto destacado, ya que puede ser controlado tanto mediante comandos seriales (UART) como a través de pines de control directos, lo que facilita su integración con diferentes sistemas embebidos.

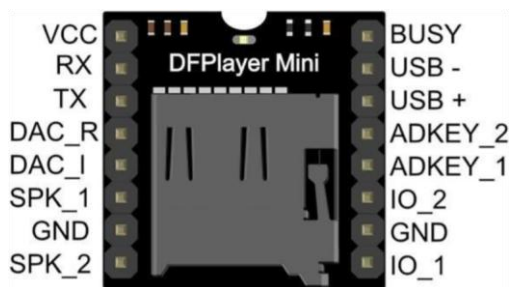
El DFPlayer Mini también cuenta con un amplificador incorporado de 3W, que permite conectar altavoces directamente al módulo sin necesidad de utilizar un amplificador externo. Además, ofrece la posibilidad de ajustar el volumen, aplicar efectos de ecualización y configurar la repetición de pistas mediante comandos, proporcionando un control total sobre la experiencia de reproducción de audio.

### Configuración de Pines

El DFPlayer Mini tiene una configuración de pines sencilla, lo que facilita su integración con microcontroladores como el ESP32 o Arduino:

- **VCC:** Alimentación del módulo (entre 3.2V y 5V).
- **GND:** Conexión a tierra.
- **RX:** Recepción de datos (comunicación serial).
- **TX:** Transmisión de datos (comunicación serial).
- **SPK1 y SPK2:** Salidas para conectar un altavoz directamente.

- **DAC\_R y DAC\_L:** Salidas de audio analógico para conectar a un amplificador externo o un sistema de audio.



*Ilustración7.PinesdelDFPlayerMini.*

## Tipos de Conexión

Existen dos formas principales de conectar el DFPlayer Mini a un sistema embebido:

### 1. Conexión mediante comunicación UART:

- Es la forma más común y flexible de controlar el DFPlayer Mini. Se conecta el pin RX del DFPlayer al pin TX del microcontrolador y el pin TX del DFPlayer al pin RX del microcontrolador.
- Mediante comandos seriales, es posible reproducir pistas específicas, pausar, ajustar el volumen y seleccionar modos de reproducción.

### 2. Conexión mediante pines de control directos:

- Se pueden utilizar los pines IO1 e IO2 del DFPlayer para reproducir pistas predefinidas sin necesidad de comunicación serial.
- Esta configuración es útil en aplicaciones simples donde no se requiere un control avanzado del audio.

## Comandos Seriales

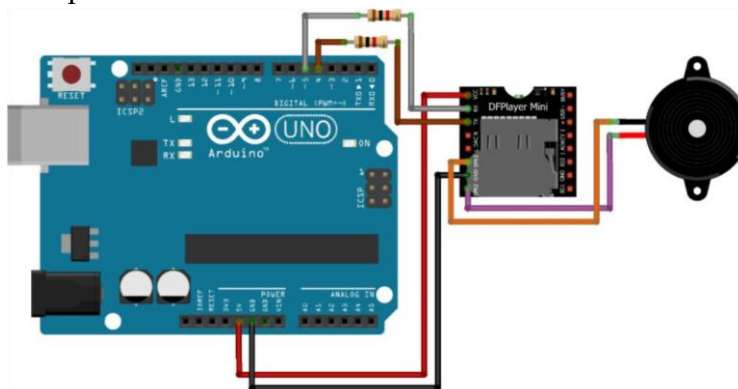
El DFPlayer Mini se controla mediante una serie de comandos enviados a través de la interfaz UART. Algunos de los comandos más comunes son:

- **0x03:** Reproducir una pista en particular.
- **0x06:** Ajustar el volumen.
- **0x0D:** Pausar la reproducción.
- **0x0E:** Reanudar la reproducción.
- **0x19:** Establecer el modo de ecualización (normal, pop, rock, jazz, clásico, etc.).

## Procedimiento de Conexión y Configuración

1. **Alimentación:** Conectar el pin VCC a una fuente de alimentación de 3.3V o 5V y el pin GND a tierra.

2. **Conexión UART:** Conectar RX y TX a los pines correspondientes del microcontrolador.
3. **Altavoz:** Si se desea salida de audio directa, conectar un altavoz a los pines SPK1 y SPK2. Alternativamente, se pueden utilizar las salidas DAC\_R y DAC\_L para conectar a un amplificador externo.



*Ilustración8.EjemplodeconexióndelDFPlayerMini.*

4. **Cargar archivos de audio:** Insertar una tarjeta microSD con los archivos de audio en formato compatible (MP3, WAV o WMA).

#### **Importancia en Sistemas Embebidos**

El DFPlayer Mini es un componente fundamental en los sistemas embebidos que requieren una salida de audio eficiente y versátil. Este módulo facilita la integración de audio en proyectos tecnológicos al eliminar la necesidad de realizar un procesamiento complejo de señales, lo que simplifica considerablemente el desarrollo. Su capacidad para leer tarjetas microSD de hasta 32 GB proporciona un almacenamiento masivo suficiente para albergar una amplia variedad de archivos de audio, adaptándose a diferentes necesidades de aplicación.

Además, el DFPlayer Mini destaca por su control flexible, ya que permite operar mediante comandos seriales o a través de pines de control directos. Esta dualidad lo hace adecuado tanto para proyectos básicos como para aplicaciones más avanzadas que demandan mayor personalización. Su versatilidad lo convierte en una solución ideal para diversos escenarios, como sistemas de aviso por voz, dispositivos IoT con retroalimentación auditiva, máquinas recreativas y proyectos interactivos. Su capacidad para adaptarse a estas aplicaciones lo posiciona como una herramienta indispensable en el diseño de sistemas embebidos con requerimientos de audio.

En esta tarea, el uso del ADC, la pantalla OLED y el módulo DFPlayer Mini se integra para construir un juguete interactivo capaz de producir sonidos, mostrar animaciones y responder a la interacción del usuario mediante botones y controles. Esto permite aplicar los principios de los sistemas embebidos a un producto lúdico y creativo.

#### **4. TAREA PRÁCTICA**

Cada estudiante o grupo deberá diseñar y desarrollar un juguete interactivo funcional que integre un microcontrolador ESP32, un módulo DFPlayer Mini y una pantalla OLED. El

juguete debe ser creativo, seguro y ofrecer una interacción significativa para el usuario mediante botones, potenciómetros u otros controles. **Ejemplos de juguetes posibles:**

- **Juguete de sonidos de animales** (cada botón reproduce un sonido distinto).
- **Mini consola retro** con sonidos y animaciones simples.
- **Juguete musical** que combine botones y potenciómetro para controlar efectos.
- **Muñeco parlante** con expresiones mostradas en la pantalla OLED.
- **Juguete educativo** que enseña números, letras o idiomas mediante audio.

**Requisitos técnicos mínimos:**

- Uso del **ESP32** como microcontrolador central.
- Integración del **DFPlayer Mini** para reproducción de sonidos.
- Implementación de una **pantalla OLED (I2C)** para mostrar información o animaciones.
- Al menos **dos botones físicos** para interacción.
- Control de **volumen con potenciómetro** (opcional pero recomendado).
- Un **menú o interfaz simple** en la pantalla OLED.
- **Diseño físico del juguete** (el modelo 3D es obligatorio, la impresión 3D es opcional).

**Entregables:**

Documento PDF con:

- Descripción del propósito del juguete.
- Capturas del diseño 3D del juguete.
- Capturas de las conexiones electrónicas.
- Enlace al repositorio GitHub con el código.

**VÍNCULO GITHUB:** <https://github.com/TheDevil21/Juguete.git>

Video en YouTube mostrando:

- Funcionamiento del juguete.
- Explicación del sistema.
- Uso del DFPlayer y OLED.

**VÍNCULO VIDEO:**

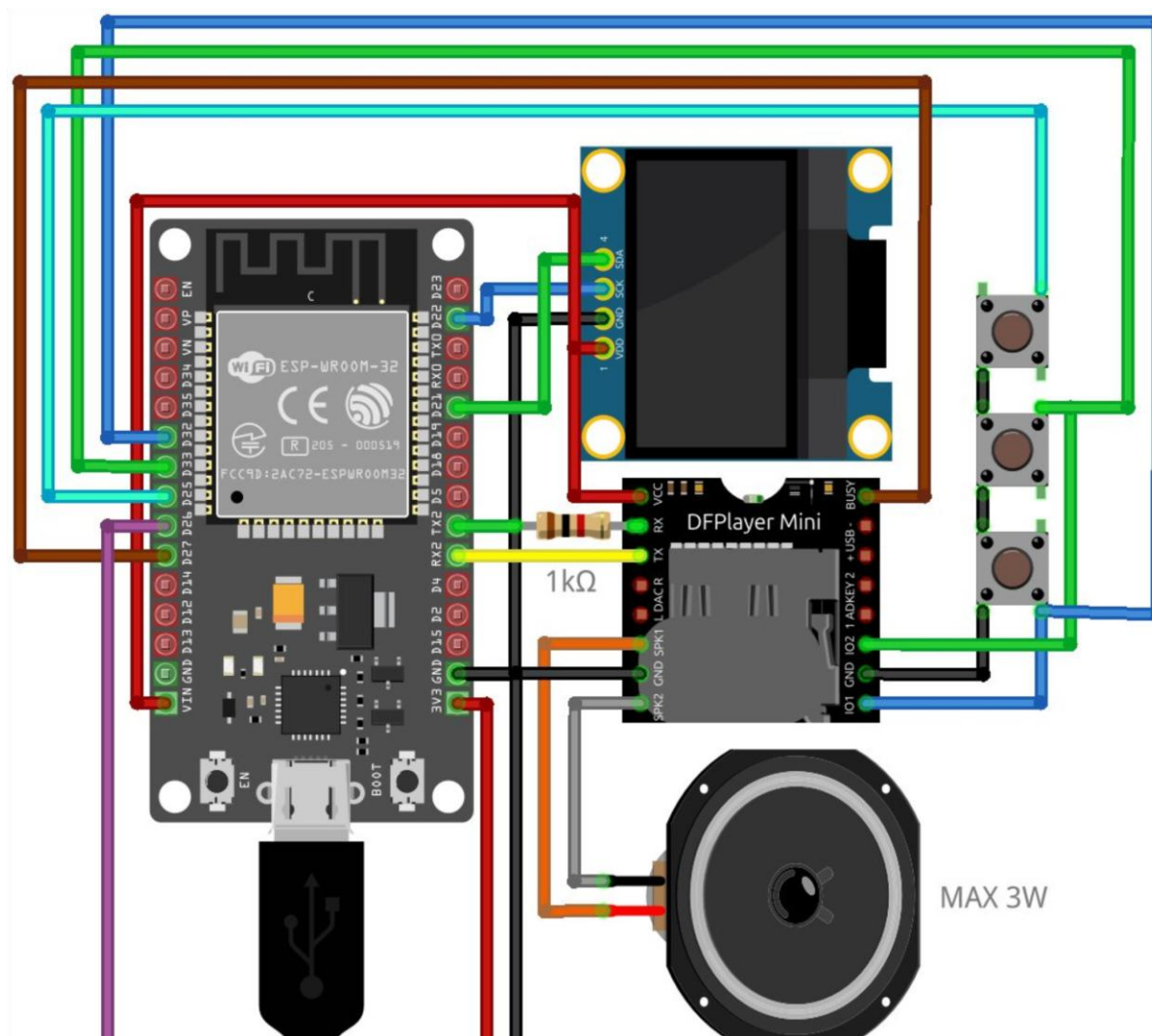
## 5. PROYECTO – Reproductor de Héroes con Animaciones OLED y DFPlayer Mini

Este proyecto demuestra cómo integrar una interfaz visual dinámica basada en animaciones de mapas de bits (bitmaps) con un sistema de audio sincronizado utilizando una ESP32 y un DFPlayer Mini. El sistema permite al usuario interactuar con diferentes "perfiles" de personajes, activando tanto contenido visual como auditivo de forma simultánea.

A continuación, se presenta el funcionamiento de este sistema interactivo que utiliza una máquina de estados para gestionar la navegación y la reproducción multimedia. Incluye las siguientes funciones:

- Reproducción temática: Ejecuta pistas de audio específicas para cada héroe almacenadas en una tarjeta microSD.
- Selección por botonera dedicada: Navegación directa mediante cuatro botones independientes, uno para cada personaje (Batman, Spiderman, Flash y Superman).
- Animaciones en mapa de bits: Visualización de secuencias de imágenes (frames) almacenadas en la memoria Flash (PROGMEM), con ajuste automático de centrado según el tamaño de cada imagen.
- Efectos visuales de transición: Incluye una "barra de carga" animada y un icono de nota musical que aparece al seleccionar un héroe para mejorar la experiencia de usuario.
- Interfaz de menú: Pantalla de inicio que invita al usuario a elegir un personaje cuando el sistema está en reposo.
- Control de interrupción: Capacidad de detener cualquier animación y música en curso para volver al menú principal mediante la presión de cualquier botón.
- Gestión eficiente de memoria: Uso de arreglos de punteros para manejar diferentes resoluciones de imagen (100x56, 64x64, 33x64) sin saturar la RAM del microcontrolador.

Diagrama esquemático de conexiones:



Explicación por secciones del código:

## 1) Inclusión de Librerías y Definición de Hardware

Se incluyen las herramientas necesarias para que el ESP32 se comunice con los periféricos:

- **DFRobotDFPlayerMini:** Para controlar la reproducción de archivos MP3.
- **Adafruit\_SSD1306 y GFX:** Para gestionar el dibujo de gráficos y texto en la pantalla OLED.
- **Wire:** Para la comunicación I2C del display.
- **Definición de Pines:** Se asignan los pines para los botones de los héroes (Batman, Spidey, Flash, Superman) y la comunicación serial con el reproductor.

```
C: > Users > Estra > Downloads > juguete > juguete > src > main.cpp
1  #include <Arduino.h>
2  #include <DFRobotDFPlayerMini.h>
3  #include <Wire.h>
4  #include <Adafruit_GFX.h>
5  #include <Adafruit_SSD1306.h>
6
7  // --- Definiciones del Display OLED ---
8  #define SCREEN_WIDTH 128
9  #define SCREEN_HEIGHT 64
10 #define OLED_RST_PIN -1
11 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RST_PIN);
12
13 // -- Configuración de DFPlayer Mini ---
14 #define MP3_RX_PIN 16
15 #define MP3_TX_PIN 17
16 #define BUSY_PIN 27
17 #define IO_1_PIN 12
18 #define IO_2_PIN 13
19 #define BTN_PAUSE_PIN 14
20 DFRobotDFPlayerMini player;
21
22 // --- CONFIGURACIÓN DE PINES (Todos con PULLUP interno) ---
23 #define PIN_BATMAN 25
24 #define PIN_SPIDERMAN 26
25 #define PIN_FLASH 32
26 #define PIN_SUPERMAN 33
27
28 // --- Configuración de Animación ---
29 #define ANIM_DELAY 80
30 #define ANIM_COUNT 8 // Nota: Superman tiene 10 frames, se cortarán los 2 últimos si dejamos esto en 8, pero funcionará.
31
32 // --- Icono de Música (Menú) ---
33 #define MUSIC_ICON_WIDTH 24
34 #define MUSIC_ICON_HEIGHT 24
```

## 2) Almacenamiento de Animaciones (Bitmaps)

Esta es la sección más extensa, donde se guardan las imágenes:

- **PROGMEM:** Se utiliza para almacenar las imágenes en la memoria Flash del ESP32 (ya que son muy pesadas para la memoria RAM).
- **Frames de Héroes:** Cada héroe tiene una serie de cuadros (frames). Batman y Spiderman usan **100x56px**, Flash **64x64px** y Superman **33x64px**.
- **Arrays de Punteros:** Se crean listas (como batmanAnim[8]) que agrupan los cuadros para que la función de animación pueda recorrerlos fácilmente en orden.

```
// BATMAN
extern const unsigned char epd_bitmapbatman_frame_0_delay_0[];
extern const unsigned char epd_bitmapbatman_frame_1_delay_0[];
extern const unsigned char epd_bitmapbatman_frame_2_delay_0[];
extern const unsigned char epd_bitmapbatman_frame_3_delay_0[];
extern const unsigned char epd_bitmapbatman_frame_4_delay_0[];
extern const unsigned char epd_bitmapbatman_frame_5_delay_0[];
extern const unsigned char epd_bitmapbatman_frame_6_delay_0[];
extern const unsigned char epd_bitmapbatman_frame_7_delay_0[];

// SPIDERMAN
extern const unsigned char epd_bitmapspecterman_frame_0_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_1_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_2_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_3_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_4_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_5_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_6_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_7_delay_0[];
extern const unsigned char epd_bitmapspecterman_frame_8_delay_0[];

// FLASH
extern const unsigned char epd_bitmapflash_frame_0_delay_0[];
extern const unsigned char epd_bitmapflash_frame_1_delay_0[];
extern const unsigned char epd_bitmapflash_frame_2_delay_0[];
extern const unsigned char epd_bitmapflash_frame_3_delay_0[];
extern const unsigned char epd_bitmapflash_frame_4_delay_0[];
extern const unsigned char epd_bitmapflash_frame_5_delay_0[];
extern const unsigned char epd_bitmapflash_frame_6_delay_0[];
extern const unsigned char epd_bitmapflash_frame_7_delay_0[];

// SUPERMAN (CORREGIDO: Nombres con '00', '01'...)
extern const unsigned char epd_bitmapsuperman_frame_0_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_1_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_2_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_3_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_4_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_5_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_6_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_7_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_8_delay_0[];
extern const unsigned char epd_bitmapsuperman_frame_9_delay_0[];
```

```
// --- ARRAYS DE PUNTEROS ---

const unsigned char* batmanAnim[8] = {
    epd_bitmapbatman_frame_0_delay_0, epd_bitmapbatman_frame_1_delay_0, epd_bitmapbatman_frame_2_delay_0, epd_bitmapbatman_frame_3_delay_0,
    epd_bitmapbatman_frame_4_delay_0, epd_bitmapbatman_frame_5_delay_0, epd_bitmapbatman_frame_6_delay_0, epd_bitmapbatman_frame_7_delay_0
};

const unsigned char* spectermanAnim[8] = {
    epd_bitmapspecterman_frame_0_delay_0, epd_bitmapspecterman_frame_1_delay_0, epd_bitmapspecterman_frame_2_delay_0, epd_bitmapspecterman_frame_3_delay_0,
    epd_bitmapspecterman_frame_4_delay_0, epd_bitmapspecterman_frame_5_delay_0, epd_bitmapspecterman_frame_6_delay_0, epd_bitmapspecterman_frame_7_delay_0
};

const unsigned char* flashAnim[8] = {
    epd_bitmapflash_frame_0_delay_0, epd_bitmapflash_frame_1_delay_0, epd_bitmapflash_frame_2_delay_0, epd_bitmapflash_frame_3_delay_0,
    epd_bitmapflash_frame_4_delay_0, epd_bitmapflash_frame_5_delay_0, epd_bitmapflash_frame_6_delay_0, epd_bitmapflash_frame_7_delay_0
};

const unsigned char* supermanAnim[10] = {
    epd_bitmapsuperman_frame_0_delay_0, epd_bitmapsuperman_frame_1_delay_0, epd_bitmapsuperman_frame_2_delay_0, epd_bitmapsuperman_frame_3_delay_0,
    epd_bitmapsuperman_frame_4_delay_0, epd_bitmapsuperman_frame_5_delay_0, epd_bitmapsuperman_frame_6_delay_0, epd_bitmapsuperman_frame_7_delay_0,
    epd_bitmapsuperman_frame_8_delay_0, epd_bitmapsuperman_frame_9_delay_0
};

// =====
```

### 3) Máquina de Estados y Variables Globales

El código utiliza un sistema organizado para saber qué está pasando:

- **enum SystemState:** Define los estados posibles: MENU, PLAY\_BATMAN, PLAY\_SPIDERMAN, etc. Esto evita que el código se confunda entre pantallas.
- **Variables de Control:** \* currentFrame: Rastrea qué cuadro de la animación se debe mostrar.
  - currentState: Almacena el estado actual del sistema.
  - isPlaying: Indica si el audio está activo.

```
// =====  
  
enum SystemState {  
    MENU,  
    PLAY_BATMAN,  
    PLAY_SPIDERMAN,  
    PLAY_FLASH,  
    PLAY_SUPERMAN  
};
```

### 4) Funciones de Animación y Efectos Visuales

- **playAnimation():** Es el "motor" gráfico. Recibe el arreglo del héroe y su tamaño, calcula el centro de la pantalla para que la imagen no se corte y dibuja el cuadro actual, avanzando al siguiente con un pequeño retraso (ANIM\_DELAY).
- **playMusicLoadingEffect():** Genera una transición visual. Muestra una barra de carga que se llena antes de mostrar al héroe, dando una sensación de "carga de sistema".
- **goToState():** Una función auxiliar que activa el efecto de carga y cambia el estado del sistema.

```
// --- FUNCIÓN DE DIBUJO MODIFICADA (AHORA ACEPTA TAMAÑO) ---
void playAnimation(const unsigned char* animArray[], int w, int h) {
    display.clearDisplay();
    // Centramos la imagen basandonos en su ancho (w) y alto (h) reales
    int x = (SCREEN_WIDTH - w) / 2;
    int y = (SCREEN_HEIGHT - h) / 2;

    // Dibujamos con el tamaño correcto
    display.drawBitmap(x, y, animArray[currentFrame], w, h, 1);

    display.display();
    currentFrame = (currentFrame + 1) % ANIM_COUNT; // Avanzamos frame
    delay(ANIM_DELAY);
}

void checkExitButton(int activePin) {
    // Si presiona CUALQUIER BOTÓN, volvemos al menú
    if (digitalRead(PIN_BATMAN) == LOW || digitalRead(PIN_SPIDERMAN) == LOW ||
        digitalRead(PIN_FLASH) == LOW || digitalRead(PIN_SUPERMAN) == LOW) {

        currentState = MENU;
        delay(500);
    }
}

void drawMenu() {
    display.clearDisplay();
    display.drawBitmap((SCREEN_WIDTH - MUSIC_ICON_WIDTH)/2, 0, smallMusicIcon, MUSIC_ICON_WIDTH, MUSIC_ICON_HEIGHT, 1);

    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(30, 25);
    display.println(F("Reproductor"));
    display.setCursor(10, 45);
    display.print(F("Elige un heroe..."));

    display.display();
}
}
```

```
void playMusicLoadingEffect(String heroName) {
    int maxBarWidth = SCREEN_WIDTH - 20;

    for(int i = 0; i <= 100; i+=4) {
        display.clearDisplay();

        display.setTextSize(1);
        display.setTextColor(SSD1306_WHITE);
        display.setCursor(20, 5);
        display.print(F("Cargando: "));
        display.println(heroName);

        int iconX = (SCREEN_WIDTH - MUSIC_ICON_WIDTH) / 2;
        int iconY = 20;
        display.drawBitmap(iconX, iconY, smallMusicIcon, MUSIC_ICON_WIDTH, MUSIC_ICON_HEIGHT, 1);

        int barX = 10;
        int barY = SCREEN_HEIGHT - 10;
        display.drawRect(barX, barY, maxBarWidth + 2, 6 + 2, SSD1306_WHITE);
        display.fillRect(barX + 1, barY + 1, (int)((float)i/100.0)*maxBarWidth, 6, SSD1306_WHITE);

        display.display();
    }
    delay(200);
}

void goToState(SystemState newState, String name) {
    playMusicLoadingEffect(name);
    currentState = newState;
    currentFrame = 0;
}

void checkMenuButtons() {
    if (digitalRead(PIN_BATMAN) == LOW) {
        goToState(PLAY_BATMAN, "Batman");
    } else if (digitalRead(PIN_SPIDERMAN) == LOW) {
        goToState(PLAY_SPIDERMAN, "Spidey");
    } else if (digitalRead(PIN_FLASH) == LOW) {
        goToState(PLAY_FLASH, "Flash");
    } else if (digitalRead(PIN_SUPERMAN) == LOW) {
        goToState(PLAY_SUPERMAN, "Superman");
    }
}
}
```

### 5) Configuración Inicial en setup()

Se preparan los componentes al encender el dispositivo:

- **Serial y Display:** Inicia la comunicación de datos y la pantalla OLED (usando la dirección 0x3C).
- **Pines de Entrada:** Se configuran los botones de los héroes con INPUT\_PULLUP (para usar los resistores internos del ESP32).
- **Limpieza:** Se borra la pantalla para iniciar desde cero en el menú.

```
void setup() {
    Serial.begin(115200);
    if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println(F("Fallo SSD1306"));
        for(;;);
    }

    pinMode(PIN_BATMAN, INPUT_PULLUP);
    pinMode(PIN_SPIDERMAN, INPUT_PULLUP);
    pinMode(PIN_FLASH, INPUT_PULLUP);
    pinMode(PIN_SUPERMAN, INPUT_PULLUP);

    pinMode(BUSY_PIN, INPUT);
    pinMode(IO_1_PIN, INPUT);
    pinMode(IO_2_PIN, INPUT);
    pinMode(BTN_PAUSE_PIN, INPUT_PULLUP);

    Serial.begin(115200);

    display.clearDisplay();
    display.display();
}
```

## 6) loop(): Lógica de Navegación y Ejecución

El bucle principal actúa como un selector de funciones mediante un switch:

- **Caso MENU:** Llama a drawMenu() y espera a que checkMenuButtons() detecte que presionaste a un héroe.
- **Casos de Reproducción (PLAY\_XXX):** \* Dibuja la animación específica del héroe con su tamaño correcto.
  - Envía el comando al **DFPlayer** para reproducir la carpeta y pista asignada (ej. Carpeta 15, pista 2 para Batman).
  - Ejecuta checkExitButton(), que está siempre atento por si presionas cualquier botón para detener todo y volver al menú principal.

## 6. CONCLUSIONES Y RECOMENDACIONES

### Conclusiones:

- **Efectividad de la Máquina de Estados:** La implementación de una estructura basada en switch-case (máquina de estados) resulta fundamental para gestionar la lógica del juguete. Esto permite que el sistema pase del estado de "Menú" al de "Animación" de forma coherente, facilitando la escalabilidad del proyecto si se desean agregar más héroes en el futuro.
- **Gestión Eficiente de Memoria con PROGMEM:** Debido a que las animaciones OLED consumen una gran cantidad de datos, el uso de la directiva PROGMEM es un éxito crítico en el diseño. Al almacenar los arreglos de bits en la memoria Flash en lugar de la RAM, se garantiza la estabilidad del ESP32, evitando reinicios inesperados por falta de memoria dinámica.
- **Sinergia Multimedia para la Interactividad:** La integración del DFPlayer Mini y la pantalla OLED permite que el juguete ofrezca una respuesta multisensorial (auditiva y visual). Esta sincronización es lo que realmente convierte a un sistema embebido simple en un juguete interactivo atractivo, ya que el usuario recibe retroalimentación inmediata de sus acciones (presionar un botón).

### Recomendaciones para el Diseño y Mejora

- **Optimización de la Calidad del Audio:** El módulo DFPlayer Mini puede generar ruido eléctrico que interfiere con la pantalla OLED o viceversa. Se recomienda colocar un capacitor electrolítico de 100μF entre los pines VCC y GND del reproductor, además de utilizar una resistencia de 1kΩ en la línea de comunicación RX para proteger el módulo y mejorar la claridad del sonido.

- **Mejora de la Fluidez Visual (I2C):** Para que las animaciones de los héroes se vean más fluidas, se recomienda configurar la velocidad de comunicación I2C a 400kHz mediante el comando `Wire.setClock;` en el `setup ()`. Esto reduce el tiempo de refresco de la pantalla y permite que el `ANIM_DELAY` pueda bajarse aún más sin que la imagen se entrecorte.
- **Robustez del Prototipo (Debounce):** En un entorno de juego real, los botones suelen recibir múltiples pulsaciones accidentales. Se recomienda implementar un "debouncing" (antirrebote) por software más robusto con los botones para evitar que el sistema regrese al menú de forma errática o salte entre héroes involuntariamente.

**ENLACE DEL REPOSITORIO DEL GITHUB:**

<https://github.com/TheDevil21/Juguete.git>